



# Journey of Go

A Complete Beginner's Guide to Go Programming

---

9 Files · 5 Topics · From Hello World to Interfaces

Go Version 1.26.4 · Windows 11

Learn by Reading Code + Running It

# Table of Contents

Follow this order — each file builds on the previous one

## CHAPTER 1 — BASICS

- 1** **hello\_world.go**  
Package, import, func main, fmt.Println — your first Go program

---

- 2** **variables.go**  
var, :=, multiple assignment, constants, zero values, fmt.Printf

---

- 3** **data\_types.go**  
int8/16/32/64, float32/64, string, rune, type conversion

---

## CHAPTER 2 — CONTROL FLOW

- 4** **conditionals.go**  
if, else if, else, switch, init statements in if

---

- 5** **loops.go**  
for loop (Go's only loop), while-style, break, continue, range

---

## CHAPTER 3 — FUNCTIONS

- 6** **functions.go**  
Basic functions, multiple returns, variadic, closures, first-class functions

---

## CHAPTER 4 — DATA STRUCTURES

- 7** **arrays\_slices.go**  
Arrays (fixed), slices (dynamic), append, copy, 2D slices

---

- 8** **maps.go**  
Key-value maps, comma-ok idiom, add/update/delete, map of slices

---

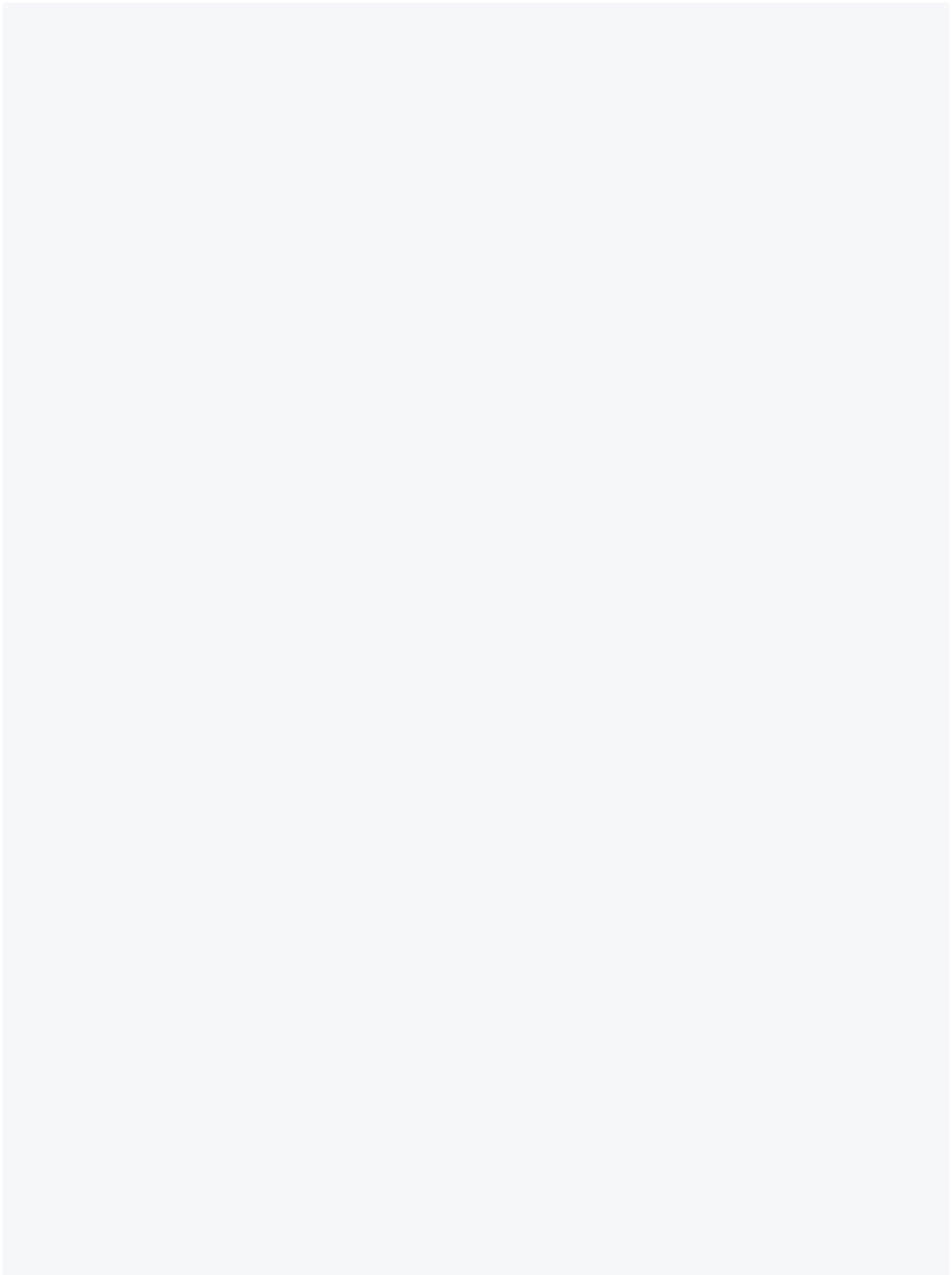
## CHAPTER 5 — STRUCTS & INTERFACES



## structs.go

Structs, methods, pointer receivers, embedding, interfaces, polymorphism

---



# Hello World — Your First Go Program

01\_basics/hello\_world.go

## The Complete Code

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, World!")
    fmt.Println("Welcome to Journey of Go!")
}
```

GO

## Line-by-Line Explanation

**package main** — Every Go file must declare which package it belongs to. The package named **main** is special: it tells Go this file is a *runnable program*, not a library. Without **package main**, you cannot run the file directly.

**import "fmt"** — Brings in the **fmt** (format) package from Go's standard library. This gives you functions for printing text, formatting strings, and reading input. Without this import, calling **fmt.Println** would cause a compile error.

**func main()** — The entry point of every Go program. When you run **go run hello\_world.go**, Go finds this function and starts executing from line 1 inside it. Every runnable Go program must have exactly one **main** function.

**fmt.Println(...)** — Calls the **Println** function from the **fmt** package. It prints the text to the terminal and automatically adds a newline at the end.

## Structure of Every Go File

| Part                | Example                       | Purpose                                                 |
|---------------------|-------------------------------|---------------------------------------------------------|
| Package declaration | <code>package main</code>     | Required. Identifies which package this file belongs to |
| Import              | <code>import "fmt"</code>     | Bring in external packages you want to use              |
| Entry point         | <code>func main()</code>      | Where execution begins (required for runnable programs) |
| Statements          | <code>fmt.Println(...)</code> | The actual work your program does                       |

## Execution Flow

```
go run hello_world.go | ▼ Go compiler reads the file | ▼ Finds:  
package main ✓ → this is a runnable program Finds: func main() ✓ →  
start here | ▼ fmt.Println("Hello, World!") → prints line 1 | ▼  
fmt.Println("Welcome to Journey of Go!") → prints line 2 | ▼ main()  
ends → program exits
```

## Output

```
$ OUTPUT  
  
Hello, World! Welcome to Journey of Go!
```

## Print Functions Comparison

| Function                   | Behaviour                       | Example                                |
|----------------------------|---------------------------------|----------------------------------------|
| <code>fmt.Println()</code> | Prints + newline at end         | <code>fmt.Println("Hi")</code>         |
| <code>fmt.Print()</code>   | Prints, no newline              | <code>fmt.Print("Hi")</code>           |
| <code>fmt.Printf()</code>  | Prints with format placeholders | <code>fmt.Printf("Hi %s", name)</code> |

## How to Run

```
# From the journey_of_go folder:  
go run 01_basics/hello_world.go  
  
# Build an .exe file:  
go build 01_basics/hello_world.go  
./hello_world.exe
```

# Variables — Storing Data in Go

01\_basics/variables.go

## The Complete Code

```
package main

import "fmt"

func main() {
    // Explicit type declaration
    var name string = "Anoop"
    var age int = 25
    var height float64 = 5.9
    var isStudent bool = true

    // Short variable declaration (most common in Go)
    city := "Delhi"
    score := 98.5

    // Multiple assignment
    x, y := 10, 20

    // Constants
    const Pi = 3.14159
    const AppName = "JourneyOfGo"

    fmt.Println("=== Variables Demo ===")
    fmt.Printf("Name: %s, Age: %d\n", name, age)
    fmt.Printf("Height: %.1f, Student: %v\n", height, isStudent)
    fmt.Printf("City: %s, Score: %.1f\n", city, score)
    fmt.Printf("x=%d, y=%d\n", x, y)
    fmt.Printf("Pi=%.5f, App=%s\n", Pi, AppName)

    // Zero values
    var a int
    var b string
    var c bool
    fmt.Printf("\nZero values: int=%d, string=%q, bool=%v\n", a, b, c)
}
```

GO

## Way 1 — var (Explicit Type)

You write the keyword `var`, the variable name, the type, and the value.

```
var  name  string  =  "Anoop"
↑    ↑    ↑      ↑
kw   name  type   value
```

| Variable               | Type                 | Value                | Stores             |
|------------------------|----------------------|----------------------|--------------------|
| <code>name</code>      | <code>string</code>  | <code>"Anoop"</code> | Text               |
| <code>age</code>       | <code>int</code>     | <code>25</code>      | Whole number       |
| <code>height</code>    | <code>float64</code> | <code>5.9</code>     | Decimal number     |
| <code>isStudent</code> | <code>bool</code>    | <code>true</code>    | true or false only |

## Way 2 — := Short Declaration (Most Common)

Go automatically figures out the type by looking at the value you assign. This is what most Go code uses.

```
city  := "Delhi"    // Go sees "Delhi" → type is string
score := 98.5       // Go sees 98.5  → type is float64
```

### RULE

`:=` can only be used *inside* a function. `var` can be used anywhere including at the package (top) level.



## | Way 3 — Multiple Assignment

```
x, y := 10, 20 // x=10 and y=20 in one line
```

You will use this constantly in Go when a function returns two values:

```
result, err := divide(10, 3) // result gets the answer, err gets any error
```

## | Constants — Values That Never Change

```
const Pi      = 3.14159
const AppName = "JourneyOfGo"

Pi = 3.0 // ❌ COMPILER ERROR: cannot assign to Pi
```

### WHEN TO USE CONST VS VAR

Use `const` for values that are always fixed: Pi, tax rates, app names, config keys. Use `var` or `:=` for values that change during the program.

## | fmt.Printf Format Verbs

Each `%_` is a placeholder replaced by the matching variable in order:

| Verb              | Type               | Example output        |
|-------------------|--------------------|-----------------------|
| <code>%s</code>   | string             | <code>Anoop</code>    |
| <code>%d</code>   | integer            | <code>25</code>       |
| <code>%f</code>   | float (6 decimals) | <code>5.900000</code> |
| <code>%.1f</code> | float (1 decimal)  | <code>5.9</code>      |
| <code>%.5f</code> | float (5 decimals) | <code>3.14159</code>  |
| <code>%v</code>   | any type           | <code>true</code>     |
| <code>%q</code>   | string with quotes | <code>" "</code>      |
| <code>\n</code>   | newline            | moves to next line    |

## | Zero Values — Go's Safety Feature

When you declare a variable without assigning a value, Go gives it a safe default automatically. No garbage values ever.

```
var a int    // a = 0
var b string // b = "" (empty string)
var c bool   // c = false
```

| Type                  | Zero Value               |
|-----------------------|--------------------------|
| <code>int</code>      | <code>0</code>           |
| <code>float64</code>  | <code>0.0</code>         |
| <code>string</code>   | <code>" "</code> (empty) |
| <code>bool</code>     | <code>false</code>       |
| pointer / slice / map | <code>nil</code>         |

## Output

```
$ OUTPUT
```

```
=== Variables Demo === Name: Anoop, Age: 25 Height: 5.9, Student: true  
City: Delhi, Score: 98.5 x=10, y=20 Pi=3.14159, App=JourneyOfGo Zero  
values: int=0, string="", bool=false
```

### IMPORTANT GO RULE

Go will NOT compile if you declare a variable but never use it. Every variable you create must be used at least once.

# Data Types — All Built-in Types

01\_basics/data\_types.go

## The Complete Code

```
package main

import (
    "fmt"
    "math"
)

func main() {
    // Integer types
    var i8  int8  = 127
    var i16 int16 = 32767
    var i32 int32 = 2147483647
    var i64 int64 = 9223372036854775807
    var u   uint  = 42

    // Float types
    var f32 float32 = 3.14
    var f64 float64 = math.Pi

    // String & rune
    var greeting string = "Namaste Go!"
    var ch rune = 'G'

    // Type conversion (always explicit in Go)
    var myInt    int    = 42
    var myFloat  float64 = float64(myInt)
    var backToInt int   = int(myFloat)

    fmt.Printf("int8:%d int16:%d int32:%d\n", i8, i16, i32)
    fmt.Printf("int64:%d uint:%d\n", i64, u)
    fmt.Printf("float32:%f\n", f32)
    fmt.Printf("float64:%.15f\n", f64)
    fmt.Printf("string:%s\n", greeting)
    fmt.Printf("rune:%c (value:%d)\n", ch, ch)
    fmt.Printf("Conversion: %d → %.1f → %d\n", myInt, myFloat, backToInt)
}
```

## Integer Types

Go has multiple integer sizes. Choose based on the range of numbers you need:

| Type               | Size                 | Range             | Use when                            |
|--------------------|----------------------|-------------------|-------------------------------------|
| <code>int8</code>  | 8 bits               | -128 to 127       | Very small numbers                  |
| <code>int16</code> | 16 bits              | -32,768 to 32,767 | Small numbers                       |
| <code>int32</code> | 32 bits              | -2.1B to 2.1B     | Most numbers                        |
| <code>int64</code> | 64 bits              | Very large        | Large numbers                       |
| <code>int</code>   | 64 bits on modern PC | Same as int64     | Default — use this most of the time |
| <code>uint</code>  | 64 bits              | 0 to very large   | When value is never negative        |

### TIP

In most Go code, just use `int`. Only use specific sizes (`int8`, `int32`, etc.) when working with binary data, files, or network protocols.

## Float Types

| Type                 | Precision          | Use when                            |
|----------------------|--------------------|-------------------------------------|
| <code>float32</code> | ~7 decimal digits  | Graphics, games (lower memory)      |
| <code>float64</code> | ~15 decimal digits | Default — use this for all decimals |

## String and Rune

```
var greeting string = "Namaste Go!" // text in double quotes
var ch rune         = 'G'           // single character in single quotes
```

| Type                | What it stores                                     | Quotes used                        |
|---------------------|----------------------------------------------------|------------------------------------|
| <code>string</code> | Text (sequence of bytes)                           | Double quotes <code>"hello"</code> |
| <code>rune</code>   | A single character (alias for <code>int32</code> ) | Single quotes <code>'G'</code>     |
| <code>byte</code>   | A single byte (alias for <code>uint8</code> )      | Single quotes <code>'A'</code>     |

## Type Conversion — Always Explicit

Go never automatically converts between types. You must do it explicitly. This prevents hidden bugs.

```
var myInt    int    = 42
var myFloat float64 = float64(myInt) // int → float64
var backInt  int    = int(myFloat)   // float64 → int (decimal part lost!)

// This would be a compile error in Go:
var x int      = 5
var y float64 = x    // ❌ ERROR - cannot use int as float64
var y float64 = float64(x) // ✅ correct
```

## Output

```
$ OUTPUT

=== Data Types in Go === -- Integers -- int8: 127, int16: 32767,
int32: 2147483647 int64: 9223372036854775807, uint: 42 -- Floats --
float32: 3.140000 float64: 3.141592653589793 -- String & Rune --
string: Namaste Go! rune: G (value: 71) string length: 11 bytes
Conversion: int(42) -> float64(42.000000) -> int(42)
```

# Conditionals — if, else, switch

02\_control\_flow/conditionals.go





```
package main

import "fmt"

func main() {
    // Basic if-else
    age := 20
    if age >= 18 {
        fmt.Println("You are an adult")
    } else {
        fmt.Println("You are a minor")
    }

    // if with init statement
    if score := 85; score >= 90 {
        fmt.Println("Grade: A")
    } else if score >= 75 {
        fmt.Println("Grade: B")
    } else if score >= 60 {
        fmt.Println("Grade: C")
    } else {
        fmt.Println("Grade: F")
    }

    // Switch statement
    day := "Monday"
    switch day {
    case "Monday", "Tuesday", "Wednesday", "Thursday", "Friday":
        fmt.Printf("%s is a weekday\n", day)
    case "Saturday", "Sunday":
        fmt.Printf("%s is a weekend\n", day)
    default:
        fmt.Println("Unknown day")
    }

    // Switch with no condition
    temp := 35
    switch {
    case temp > 40:
        fmt.Println("Very hot!")
    case temp > 30:
        fmt.Println("Hot day")
    default:
        fmt.Println("Pleasant")
    }
}
```

```
}  
  
}
```

## Basic if / else if / else

```
if age >= 18 {  
    // runs when condition is true  
} else if age >= 13 {  
    // runs when first is false, this is true  
} else {  
    // runs when all above are false  
}
```

### GO RULE

The opening `{` must be on the same line as `if` or `else`. Putting it on a new line is a compile error.

## if with Init Statement — Go's Unique Feature

```
if score := 85; score >= 90 {  
    ↑ init statement ↑    ↑ condition ↑  
    fmt.Println("Grade A")  
}
```

The variable `score` is declared and only exists inside this `if` block. It disappears after the block ends. This keeps your code clean — temporary variables don't pollute the surrounding scope.

## | Switch Statement

Switch in Go is cleaner than other languages — no `break` needed at the end of each case (Go breaks automatically).

| Feature                     | Go                              | Other languages      |
|-----------------------------|---------------------------------|----------------------|
| break needed?               | No — automatic                  | Yes (C, Java, JS)    |
| Multiple values in one case | <code>case "Mon", "Tue":</code> | Needs multiple cases |
| Switch with no condition    | Yes (acts like if-else)         | No                   |

## | Output

```
$ OUTPUT
```

```
You are an adult Grade: B Monday is a weekday Hot day
```

# Loops — Go Has Only One: for

02\_control\_flow/loops.go

## The Complete Code

GO

```
package main

import "fmt"

func main() {
    // Classic for loop
    for i := 1; i <= 5; i++ {
        fmt.Printf("i = %d\n", i)
    }

    // While-style (condition only)
    count := 0
    for count < 3 {
        fmt.Printf("count = %d\n", count)
        count++
    }

    // Infinite loop with break
    n := 0
    for {
        if n == 3 { break }
        fmt.Printf("n = %d\n", n)
        n++
    }

    // continue - skip even numbers
    for i := 1; i <= 6; i++ {
        if i%2 == 0 { continue }
        fmt.Printf("odd: %d\n", i)
    }

    // Range over slice
    fruits := []string{"Apple", "Mango", "Banana"}
    for index, value := range fruits {
        fmt.Printf("[%d] = %s\n", index, value)
    }

    // Range over map
    capitals := map[string]string{"India": "Delhi", "Japan": "Tokyo"}
    for country, capital := range capitals {
        fmt.Printf("%s → %s\n", country, capital)
    }
}
```

## 4 Ways to Use the for Loop

| Style       | Syntax                                 | Equivalent to          |
|-------------|----------------------------------------|------------------------|
| Classic     | <code>for i := 0; i &lt; 5; i++</code> | C-style for loop       |
| While-style | <code>for count &lt; 3</code>          | while loop             |
| Infinite    | <code>for { }</code>                   | while(true) / do-while |
| Range       | <code>for i, v := range items</code>   | foreach loop           |

## break and continue

```
break    // exits the loop immediately
continue // skips the rest of current iteration, goes to next
```

## Range — Most Used Loop in Go

```
// Over a slice - gives index and value
for index, value := range fruits { }

// Skip index (use _ to ignore)
for _, value := range fruits { }

// Skip value (only index)
for index := range fruits { }

// Over a map - gives key and value
for key, value := range capitals { }

// Over a string - gives index and rune (character)
for i, ch := range "Hello" { }
```

## Output

```
$ OUTPUT
```

```
i = 1 i = 2 i = 3 i = 4 i = 5 count = 0 count = 1 count = 2 n = 0 n =  
1 n = 2 odd: 1 odd: 3 odd: 5 [0] = Apple [1] = Mango [2] = Banana  
India → Delhi Japan → Tokyo
```

# Functions — Go's Superpowers

03\_functions/functions.go



## The Complete Code

```
package main

import ("fmt"; "math")

// Basic function
func greet(name string) string {
    return "Hello, " + name + "!"
}

// Multiple return values - a Go superpower
func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("cannot divide by zero")
    }
    return a / b, nil
}

// Named return values
func minMax(nums []int) (min, max int) {
    min, max = nums[0], nums[0]
    for _, n := range nums[1:] {
        if n < min { min = n }
        if n > max { max = n }
    }
    return // naked return
}

// Variadic - accepts any number of args
func sum(nums ...int) int {
    total := 0
    for _, n := range nums { total += n }
    return total
}

// First-class: function as parameter
func apply(nums []int, fn func(int) int) []int {
    result := make([]int, len(nums))
    for i, n := range nums { result[i] = fn(n) }
    return result
}

// Closure
func makeCounter() func() int {
    count := 0
    return func() int { count++; return count }
}
```

```

func main() {
    fmt.Println(greet("Anoop"))

    result, err := divide(10, 3)
    if err != nil { fmt.Println("Error:", err) } else {
        fmt.Printf("10/3 = %.4f\n", result)
    }

    min, max := minMax([]int{3, 1, 7, 2, 9})
    fmt.Printf("Min:%d Max:%d\n", min, max)

    fmt.Println(sum(1, 2, 3, 4, 5))

    squares := apply([]int{1,2,3,4,5}, func(n int) int { return n * n })
    fmt.Println("Squares:", squares)

    circleArea := func(r float64) float64 { return math.Pi * r * r }(5.0)
    fmt.Printf("Circle area: %.2f\n", circleArea)

    counter := makeCounter()
    fmt.Println(counter(), counter(), counter())
}

```

## Function Anatomy

```

func  greet  (name string)  string  { ... }
↑      ↑           ↑           ↑
kw   name    parameters  return type

```

## Multiple Return Values — Go's Superpower

Go functions can return more than one value. This is used everywhere in Go — the standard pattern is to return a result and an error:

```
result, err := divide(10, 3)
if err != nil {
    // something went wrong
}
```

### GO ERROR HANDLING PHILOSOPHY

Go does not use exceptions (try/catch). Errors are just values returned from functions. You check them explicitly with `if err != nil`. This makes error handling visible and intentional.

## Variadic Functions

```
func sum(nums ...int) int // ... means "zero or more ints"

sum(1, 2)                // works
sum(1, 2, 3, 4, 5)       // works too
sum()                    // zero args also works
```

## Closures — Functions that Remember

A closure is a function that captures and remembers variables from its surrounding scope, even after the outer function has returned.

```
func makeCounter() func() int {  
    count := 0           // this variable is "captured"  
    return func() int {  
        count++          // remembers and modifies count  
        return count  
    }  
}  
  
counter := makeCounter()  
counter() // → 1  
counter() // → 2  
counter() // → 3
```

## Output

```
$ OUTPUT  
  
Hello, Anoop! 10 / 3 = 3.3333 Error: cannot divide by zero Min: 1,  
Max: 9 Sum(1..5): 15 Squares: [1 4 9 16 25] Circle area (r=5): 78.54  
Counter: 1, 2, 3
```

# Arrays & Slices — Collections of Values

04\_data\_structures/arrays\_slices.go

## The Complete Code

```
package main

import "fmt"

func main() {
    // Array — fixed size
    var arr [5]int = [5]int{10, 20, 30, 40, 50}
    fmt.Println("Array:", arr)

    // Slice — dynamic, built on top of array
    fruits := []string{"Apple", "Mango", "Banana", "Cherry"}
    fmt.Println(fruits[1:3]) // [Mango Banana]
    fmt.Println(fruits[:2])  // [Apple Mango]
    fmt.Println(fruits[2:])  // [Banana Cherry]

    // Append to slice
    fruits = append(fruits, "Grape", "Kiwi")

    // make() — create with length and capacity
    nums := make([]int, 3, 10)

    // Copy slice
    src := []int{1, 2, 3, 4, 5}
    dst := make([]int, len(src))
    copy(dst, src)

    // 2D slice (matrix)
    matrix := [][]int{{1,2,3}, {4,5,6}, {7,8,9}}
    for _, row := range matrix { fmt.Println(row) }

    _ = nums
}
```

GO

## Array vs Slice

| Feature       | Array                    | Slice                     |
|---------------|--------------------------|---------------------------|
| Size          | Fixed at compile time    | Dynamic — grows as needed |
| Declaration   | <code>[5]int{...}</code> | <code>[]int{...}</code>   |
| Common in Go? | Rarely                   | Yes — used everywhere     |
| Built on      | Direct memory            | Array underneath          |

## Slice Operations

```
fruits := []string{"Apple", "Mango", "Banana", "Cherry"}
//           [0]       [1]       [2]       [3]

fruits[1:3] // → [Mango Banana]    from index 1 up to (not including) 3
fruits[:2]  // → [Apple Mango]     from start up to 2
fruits[2:]  // → [Banana Cherry]   from 2 to end
fruits[:]   // → all elements
```

## append — Adding Elements

```
fruits = append(fruits, "Grape")           // add one
fruits = append(fruits, "Grape", "Kiwi")   // add multiple
fruits = append(fruits, more...)           // add another slice
```

### IMPORTANT

Always assign the result of `append` back: `fruits = append(fruits, ...)`.  
`append` may create a new underlying array, so the original variable must be updated.

## make() — Create Slice with Preset Size

```
nums := make([]int, 3, 10)
//           type len cap
// len=3 → 3 elements ready (all 0)
// cap=10 → space for 10 before re-allocation
```



# Maps — Key-Value Storage

04\_data\_structures/maps.go

## The Complete Code

```
package main

import "fmt"

func main() {
    // Create a map
    capitals := map[string]string{
        "India":  "New Delhi",
        "Japan":  "Tokyo",
        "Germany": "Berlin",
    }

    // Access a value
    fmt.Println(capitals["India"])

    // Comma-ok idiom - check if key exists
    capital, ok := capitals["France"]
    if ok {
        fmt.Println("France:", capital)
    } else {
        fmt.Println("France not found")
    }

    // Add and update
    capitals["France"] = "Paris" // add
    capitals["India"]  = "Delhi"  // update

    // Delete a key
    delete(capitals, "Germany")

    // Iterate
    for country, cap := range capitals {
        fmt.Printf("%s → %s\n", country, cap)
    }

    // make() for maps
    scores := make(map[string]int)
    scores["Alice"] = 95
    scores["Bob"]   = 87
}
```

## | Map Syntax

```
map[KeyType]ValueType{ key: value, ... }  
    ↑           ↑  
    type of key  type of value
```

## | Comma-Ok Idiom — Safe Key Lookup

Accessing a missing key doesn't crash — it returns the zero value. Use comma-ok to tell the difference between "key not found" and "key found but value is zero":

```
capital, ok := capitals["France"]  
//   ↑      ↑  
// value  true if found, false if not  
  
if ok {  
    // key exists  
} else {  
    // key not in map  
}
```

## Map Operations Summary

| Operation    | Syntax                                                     |
|--------------|------------------------------------------------------------|
| Create       | <code>m := map[string]int{} or make(map[string]int)</code> |
| Add / Update | <code>m["key"] = value</code>                              |
| Read         | <code>v := m["key"]</code>                                 |
| Safe read    | <code>v, ok := m["key"]</code>                             |
| Delete       | <code>delete(m, "key")</code>                              |
| Length       | <code>len(m)</code>                                        |
| Iterate      | <code>for k, v := range m { }</code>                       |

### NOTE

Map iteration order is random in Go. Every time you range over a map, the order of keys may be different. Do not rely on any specific order.

# Structs, Methods & Interfaces

05\_structs\_interfaces/structs.go

## The Complete Code

```
package main

import ("fmt"; "math")

// Struct definition
type Person struct {
    Name string
    Age  int
    City string
}

// Value receiver method
func (p Person) Greet() string {
    return fmt.Sprintf("Hi, I'm %s from %s", p.Name, p.City)
}

// Pointer receiver - can modify the struct
func (p *Person) Birthday() { p.Age++ }

// Embedding - composition over inheritance
type Employee struct {
    Person
    CompanyName string
    Salary      float64
}

// Interface
type Shape interface {
    Area()      float64
    Perimeter() float64
}

type Circle struct { Radius float64 }
func (c Circle) Area() float64      { return math.Pi * c.Radius * c.Radius }
func (c Circle) Perimeter() float64 { return 2 * math.Pi * c.Radius }

type Rectangle struct { Width, Height float64 }
func (r Rectangle) Area() float64      { return r.Width * r.Height }
func (r Rectangle) Perimeter() float64 { return 2 * (r.Width + r.Height) }

func printShapeInfo(s Shape) {
    fmt.Printf("Area: %.2f Perimeter: %.2f\n", s.Area(), s.Perimeter())
}

func main() {
    p1 := Person{Name: "Anoop", Age: 25, City: "Delhi"}
```

```

    fmt.Println(p1.Greet())
    p1.Birthday()
    fmt.Printf("After birthday: age=%d\n", p1.Age)

    emp := Employee{
        Person:      Person{Name: "Raj", Age: 30, City: "Mumbai"},
        CompanyName: "TechCorp", Salary: 75000,
    }
    fmt.Println(emp.Greet()) // promoted from Person

    printShapeInfo(Circle{5})
    printShapeInfo(Rectangle{4, 6})
}

```

## Struct — Custom Data Type

A struct groups related fields together into one type. Think of it like a class in other languages, but simpler.

```

type Person struct { // define the struct
    Name string
    Age  int
}

p := Person{Name: "Anoop", Age: 25} // create an instance
fmt.Println(p.Name) // access fields with dot notation

```

## Methods — Value vs Pointer Receiver

| Receiver | Syntax                                   | Can modify struct?      | Use when              |
|----------|------------------------------------------|-------------------------|-----------------------|
| Value    | <code>func (p Person) Greet()</code>     | No — gets a copy        | Just reading fields   |
| Pointer  | <code>func (p *Person) Birthday()</code> | Yes — modifies original | Need to change fields |



## | Embedding — Composition over Inheritance

Go has no inheritance. Instead you embed one struct inside another. The inner struct's methods are automatically promoted to the outer struct.

```
type Employee struct {
    Person          // embedded - no field name
    CompanyName string
}

emp.Greet() // works! Greet() is promoted from Person
emp.Name    // works! Name field promoted from Person
```

## | Interface — Define Behaviour

An interface is a contract. Any type that has the required methods automatically satisfies the interface — no explicit declaration needed (implicit, not explicit like Java/C#).

```
type Shape interface {
    Area()      float64
    Perimeter() float64
}

// Circle has both Area() and Perimeter() → satisfies Shape automatically
// Rectangle has both too → also satisfies Shape

func printShapeInfo(s Shape) { // accepts any Shape
    s.Area()                  // polymorphism
    s.Perimeter()
}
```

## | Output

```
$ OUTPUT

Hi, I'm Anoop from Delhi, age 25 After birthday: age = 26 Hi, I'm Raj
from Mumbai, age 30 Raj works at TechCorp earning 75000/month Circle:
Area: 78.54, Perimeter: 31.42 Rectangle: Area: 24.00, Perimeter: 20.00
All shapes: [0] Area=28.27 [1] Area=16.00 [2] Area=153.94
```

## APPENDIX

# Quick Reference Card

Everything in one place

## Variables

```
var x int = 10    // explicit type
x := 10           // short declaration (inside func only)
x, y := 1, 2      // multiple
const Pi = 3.14   // constant
```

## Types

```
int  int8  int16  int32  int64
uint  float32  float64
string bool  rune  byte
```

## Control Flow

```
if x > 0 { } else if x < 0 { } else { }
switch x { case 1: ... default: ... }
for i := 0; i < 10; i++ { }
for condition { }
for { break }
for i, v := range slice { }
```

## Functions

```
func name(param type) returnType { return val }
func name() (int, error) { }           // multiple returns
func name(nums ...int) { }           // variadic
```

## Data Structures

```
// Slice
s := []int{1, 2, 3}
s = append(s, 4)

// Map
m := map[string]int{"a": 1}
v, ok := m["a"]
delete(m, "a")

// Struct
type T struct { Field int }
t := T{Field: 1}

// Interface
type I interface { Method() string }
```

## Format Verbs Cheatsheet

| Verb              | Meaning       | Verb            | Meaning       |
|-------------------|---------------|-----------------|---------------|
| <code>%d</code>   | integer       | <code>%s</code> | string        |
| <code>%f</code>   | float         | <code>%v</code> | any type      |
| <code>%.2f</code> | float 2 dec   | <code>%q</code> | quoted string |
| <code>%t</code>   | bool          | <code>%c</code> | character     |
| <code>%T</code>   | type of value | <code>\n</code> | newline       |



**Journey of Go — Complete**

Run each file: `go run 01_basics/hello_world.go`